SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 응용

탐욕 알고리즘

Python

학습시작

아래 문장은 컴퓨터가 탐욕 알고리즘을 생각하는 예시입니다.

1. 문제를 풀 때마다 [ ] 선택을 합니다. 왜냐하면 매 순간의 선택이 결국 전체 문제의 [ ]라고 믿기 때문입니다. 이런 방식이 잘 통하려면 [ ] 속성이 있어야 합니다.
2. 항상 [ ] 기준을 대상으로 정렬하고, 그 중에서 [ ]만을 반복합니다. 하지만 매번 성공하는 것은 아닙니다. 어떤 문제는 [ ]를 만족하지 않으면 틀릴 수 있습니다.

- 다양한 문제를 분석하며 효율적인 전략을 스스로 세울 수 있습니다.
- 특정 문제가 Greedy가 정답인지 아닌지 판단하는 기준을 가질 수 있습니다.
- 문제마다 정렬 기준을 스스로 설정하고, 반복 구조를 설계하는 능력이 향상됩니다.

학습목표

- 탐욕 기법의 개념과 특성에 대해 학습한다.
- 탐욕적 선택이 항상 최적해를 찾는다고 보장할 수 없음을 이해한다.
- 최적화 문제가 탐욕적 선택 속성과 최적 부분구조를 가질 때 탐욕 기법이 적용 가능하다는 것을 이해한다.
- 동전 거스름돈 문제, 배낭 문제, 활동 선택 문제에 대해 알아보고 탐욕 알고리즘을 적용해 본다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

조합적 문제

- 부분집합
- 조합

탐욕 알고리즘

- Greedy
- Knapsack 문제
- 활동 선택 문제
- 연습 문제

조합적 문제

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

부분집합

부분집합(Powerset)이란?

부분집합

powerset

어떤 집합의 공집합과 자기 자신을 포함한 모든 부분

구하고자 하는 어떤 집합의 원소 개수가 n일 경우 부분집합의 수는 2^n개이다.

집합에 포함된 원소들을 선택하는 것

- 부분집합 예시
  - 오른쪽 표는 집합 {A, B, C}로 만들 수 있는 부분집합의 예시입니다.
  - 부분집합에는 아무것도 선택하지 않은 경우도 포함됩니다. (공집합)

{A, B, C}
{}
{A}
{B}
{A, B}
{C}
{A, C}
{B, C}
{A, B, C}

<표1_알고리즘 응용_탐욕 알고리즘_부분집합 예시>

집합에서 부분집합을 찾아내는 구현 방법

1. 완전탐색
   - 재귀호출을 이용한 완전탐색으로 부분집합을 구할 수 있습니다.
   - 실전 보다는 완전 탐색 학습용으로 추천하는 방법입니다.

2. Binary Counting
   - 2진수와 비트연산을 이용하여 부분집합을 구할 수 있습니다.
   - 모든 부분집합이 필요할 때 사용하는 추천 방법입니다.

완전탐색으로 부분집합 구하기

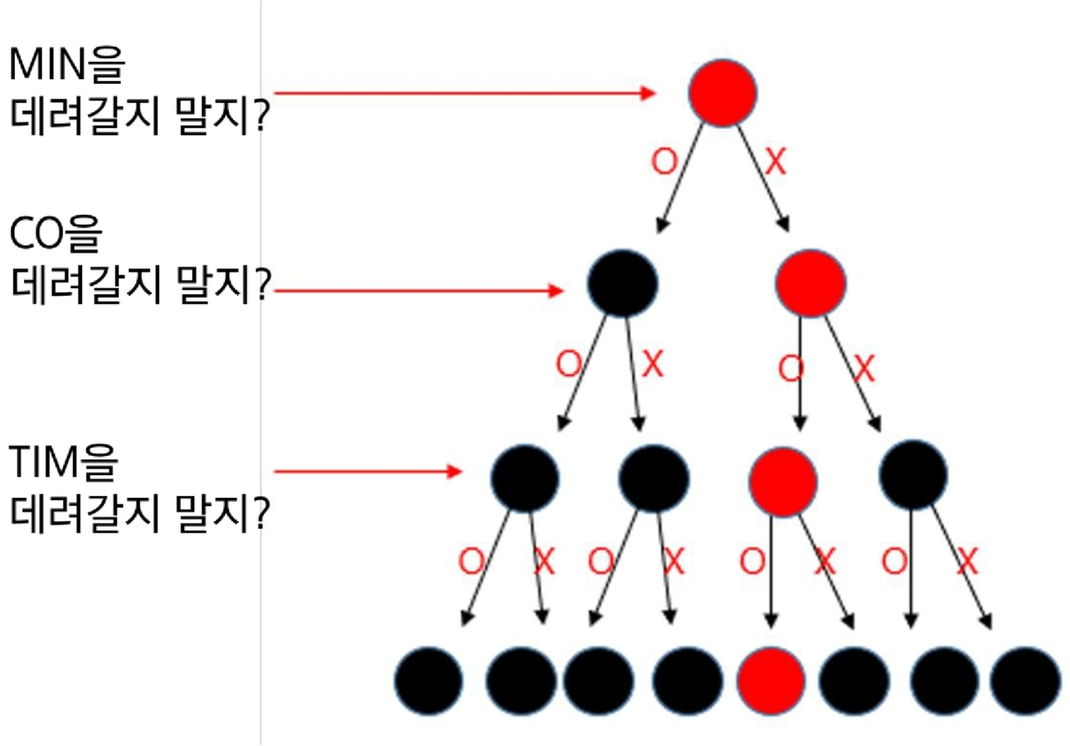
- 민철이에게는 세 명의 친구가 있습니다.
  {MIN, CO, TIM}

- 함께 영화관에 갈 수 있는 멤버를 구성하고자 합니다.

- 모든 경우의 수를 출력해봅시다.

완전 탐색을 이용해 구현

- O, X 로 집합에 포함시킬지 말지 결정



<그림1_알고리즘 응용_탐욕 알고리즘_완전탐색>

※ 이 경우 Path 배열은 [X, O, O]가 되며, 이는 MIN을 제외하고 {CO, TIM}이 선택된 것입니다.

코드 구현

- Branch : 2개
- Level : 3개

```
arr = ['O', 'X']
path = []
name = ['MIN', 'CO', 'TIM']


def run(lev):
    if lev == 3:
        print(path)
        return

    for i in range(2):
        path.append(arr[i])
        run(lev + 1)
        path.pop()


run(0)
```

```
['O', 'O', 'O']
['O', 'O', 'X']
['O', 'X', 'O']
['O', 'X', 'X']
['X', 'O', 'O’]
['X', 'O', 'X']
['X', 'X', 'O']
['X', 'X', 'X']
```

※ 친구 이름을 출력하는 코드를 추가해보세요.

완성된 소스코드

- 이름 출력 코드 추가

- 코드 : 1. subset_recursion.py

```
arr = ['O', 'X']
path = []
name = ['MIN', 'CO', 'TIM']


def run(lev):
    # 3개를 뽑았을 때 출력
    if lev == 3:
        print_name()
        return

    for i in range(2):
        path.append(arr[i])
        run(lev + 1)
        path.pop()


run(0)
```

```
# 이름 출력 함수
def print_name():
    print('{ ', end='')
    for i in range(3):
        if path[i] == 'O':
            print(name[i], end=' ')
    print('}')
```

```
{ MIN CO TIM }
{ MIN CO }
{ MIN TIM }
{ MIN }
{ CO TIM }
{ CO }
{ TIM }
{ }
```

바이너리 카운팅(Binary Counting)

- 원소 수에 해당하는 N개의 비트열을 이용해 부분집합을 표시
- 001 이면 부분집합 {A}를 나타냄

 - 0번 비트가 1이므로 첫 원소인 A만 포함된 부분집합을 나타냅니다.

- 110 이면 부분집합 {B, C}를 나타냄

 - 1번, 2번 비트가 1이므로, 두 번째와 세 번째 원소인 B, C가 포함된 부분집합을 나타냅니다.

10진수	이진수	{A, B, C}
0	000	{}
1	001	{A}
2	010	{B}
3	011	{A, B}
4	100	{C}
5	101	{A, C}
6	110	{B, C}
7	111	{A, B, C}

<표2_알고리즘 응용_탐욕 알고리즘_바이너리 카운팅>

부분집합의 총 개수

- 만들 수 있는 집합의 총 개수는 2^n 이며,

  n = 3 이기에 총 8개의 부분집합 존재

- 2^n 은 1 << n 공식을 이용하여 빠르게 구할 수 있음

```
print(pow(2,3))
print(1<<3)
```

```
8
8
```

10진수	이진수	{A, B, C}
0	000	{}
1	001	{A}
2	010	{B}
3	011	{A, B}
4	100	{C}
5	101	{A, C}
6	110	{B, C}
7	111	{A, B, C}

<표2_알고리즘 응용_탐욕 알고리즘_바이너리 카운팅>

부분집합 {B, C}를 만드는 과정

- 6 (0b110)에서 비트 연산을 이용하여 마지막 한 자리가 1인지 0인지 검사

```
arr = ['A', 'B', 'C']
n = len(arr)

def get_sub(tar):
    for i in range(n):
        if tar & 0x1:
            print(arr[i], end='')
        tar >>= 1
get_sub(6)
```

- 검사한 한 자리를 제거 (tar >>= 1)

10진수	이진수	{A, B, C}
0	000	{}
1	001	{A}
2	010	{B}
3	011	{A, B}
4	100	{C}
5	101	{A, C}
6	110	{B, C}
7	111	{A, B, C}

<표2_알고리즘 응용_탐욕 알고리즘_바이너리 카운팅>

완성된 부분집합 코드

- get_sub(0) ~ get_sub(7) 까지 호출하여 모든 부분집합을 출력

```
arr = ['A', 'B', 'C']
n = len(arr)

def get_sub(tar):
    for i in range(n):
        if tar & 0x1:
            print(arr[i], end='')
        tar >>= 1

for tar in range(1 << n):    # range(0, 8)
    print('{', end='')
    get_sub(tar)
    print('}')
```

```
{}
{A}
{B}
{AB}
{C}
{AC}
{BC}
{ABC}
```

- 코드 : 2.subset_binary.py

10진수	이진수	{A, B, C}
0	000	{}
1	001	{A}
2	010	{B}
3	011	{A, B}
4	100	{C}
5	101	{A, C}
6	110	{B, C}
7	111	{A, B, C}

<표2_알고리즘 응용_탐욕 알고리즘_바이너리 카운팅>

친구와 카페 방문

민철이는 친구 {A, B, C, D, E}가 있습니다.

이 중 최소 2명 이상의 친구와 함께 카페에 가려고 합니다.

<그림2_알고리즘 응용_탐욕 알고리즘_친구와 카페 방문 문제>

총 몇 가지 경우가 가능할까요?
(제한시간 5분)

- 코드 : 3.problem_go_cafe_with_friends.py

먼저, 바이너리 카운팅 소스코드를 복습합니다.
이후, 소스코드를 수정하여 직접 구현해보세요.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

조합

조합(Combination)이란?

조합

Combination

서로 다른 n개의 원소 중 r개를 순서 없이 골라낸 것

순열과 조합 차이

순열

{A, B, C, D, E} 5명 중 1등, 2등, 3등 뽑기

- A B C 와 C B A 는 다른 경우이다.

$$_nP_r = \frac{n!}{(n-r)!}\ ,\ (n \geq r)$$

조합

5명 중 3명 뽑기

- A B C 와 C B A 는 같은 경우이다.

$$nCr = \frac{n!}{(n-r)!\,r!}, (n \geq r)$$

$$nCr = {}_{n-1}C_{r-1} + {}_{n-1}C_r$$

$$_nC_0 = 1$$

[도전] {A, B, C, D, E} 5명 중 3명 뽑을 수 있는 모든 경우를 메모장에 적어보자. (제한시간 2분)

힌트 : ABC ~ CDE

- 메모장에 모든 경우를 적으면서, 규칙을 찾아보세요.
- 코드 : 4.combination_recursion.py

만약 5명 중 3명을 뽑는 코드는 몇 중 for문이 필요할까?

→ 3중 for문으로 구현이 가능

만약 5명 중 n명을 뽑는 코드는 몇 중 for문이 필요할까?

→ N중 for문으로 구현이 가능. 즉, 재귀호출 구현이 필요

→ Branch : 최대 5개

→ Level : n

코드에 start 파라미터를 추가하여 조합 소스코드를 완성한다.

Level = n

Branch = 5

1. 처음 recur 함수의 start 값은 0이다. 따라서 0 ~ 5까지 반복하면서 재귀호출을 한다.

2. 만약 i가 3이 선택되는 경우는 재귀 호출할 때 start는 4가 된다. 다음 for문은 4부터 수행한다.

```
arr = ['A', 'B', 'C', 'D', 'E']
n = 3

path = []

def recur(cnt, start):
    if cnt == n:
        print(*path)
        return

    for i in range(start, len(arr)):
        path.append(arr[i])
        recur(cnt + 1, i + 1)
        path.pop()

recur(0, 0)
```

[도전] 주사위 던지기

- 주사위 N개를 던져서 나올 수 있는 눈금의 모든 조합을 출력하기 (구현)

- N = 3 일 때 출력 결과
(제한시간 7분)
- 코드 : 5.dice.py

<그림3_알고리즘 응용_탐욕 알고리즘_주사위 던지기 문제>

[1,1,1] [1,1,2] [1,1,3] [1,1,4] [1,1,5] [1,1,6]	[1,2,2] [1,2,3] [1,2,4] [1,2,5] [1,2,6]	[1,3,3] [1,3,4] [1,3,5] [1,3,6]	[1,4,4] [1,4,5] [1,4,6] [1,5,5] [1,5,6] [1,6,6]	[2,2,2] [2,2,3] [2,2,4] [2,2,5] [2,2,6]	[2,3,3] [2,3,4] [2,3,5] [2,3,6]	[2,4,4] [2,4,5] [2,4,6] [2,5,5] [2,5,6] [2,6,6]	[3,3,3] [3,3,4] [3,3,5] [3,3,6]	[3,4,4] [3,4,5] [3,4,6] [3,5,5] [3,5,6] [3,6,6]	[4,4,4] [4,4,5] [4,4,6] [4,5,5] [4,5,6] [4,6,6]	[5,5,5] [5,5,6] [5,6,6] [6,6,6]

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

탐욕 알고리즘

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Greedy

Greedy(욕심쟁이 기법, 알고리즘)란?

Greedy

탐욕 알고리즘

결정이 필요할 때, 현재 기준으로 가장 좋아 보이는
선택지로 결정하여 답을 도출하는 알고리즘

대표적인 문제해결 기법

1. 완전 탐색 (Brute-Force)
   - 답이 될 수 있는 모든 경우를 시도해보는 알고리즘
2. Greedy
   - 결정이 필요할 때 가장 좋아 보이는 선택지로 결정하는 알고리즘
3. DP
   - 현재에서 가장 좋아 보이는 것을 선택하는 것이 아닌, 과거의 데이터를 이용하여 현재의 데이터를 만들어내는 문제해결 기법
4. 분할 정복 (차후 학습 예정)
   - 큰 문제를 작은 문제로 나누어 해결하는 문제해결 기법

동전 교환 문제

아래 그림과 같이 총 네 종류의 동전이 있다.
손님의 돈을 최소한의 동전 수를 사용하여 교환해 주려고 한다.

만약 1,730원을 거슬러주기 위해 사용할 수 있는 최소 동전 수는 몇 개인가?

<그림4_알고리즘 응용_탐욕 알고리즘_동전 교환1>

1,730원을 거슬러 줄 수 있는 최소 동전의 수

- 큰 동전부터 최대한 거슬러 주면 된다.
  이처럼, 좋아 보이는 값을 먼저 선택하는 것을 그리디(Greedy), 탐욕(욕심쟁이) 알고리즘이라고 한다.

- 500원 = 3개 = 누적 1,500원
- 100원 = 2개 = 누적 1,700원
- 10원 = 3개 = 누적 1,730원

<그림4_알고리즘 응용_탐욕 알고리즘_동전 교환1>

그리디로 성립하는 조건인 10, 50, 100, 500원 동전일 때 1,730원을 거슬러 주는 소스 코드

```
coin_list = [500, 100, 50, 10]  # 큰 동전부터 앞으로 작성함
target = 1730
cnt = 0

for coin in coin_list:
    possible_cnt = target // coin   # 현재 동전으로 가능한 최대 수

    cnt += possible_cnt             # 정답에 더해준다.
    target -= coin * possible_cnt   # 금액을 빼준다.
print(cnt)
```

- 출력 결과 : 8
  - 코드 : 6.problem_coin_swap.py

동전 구성이 다른 경우

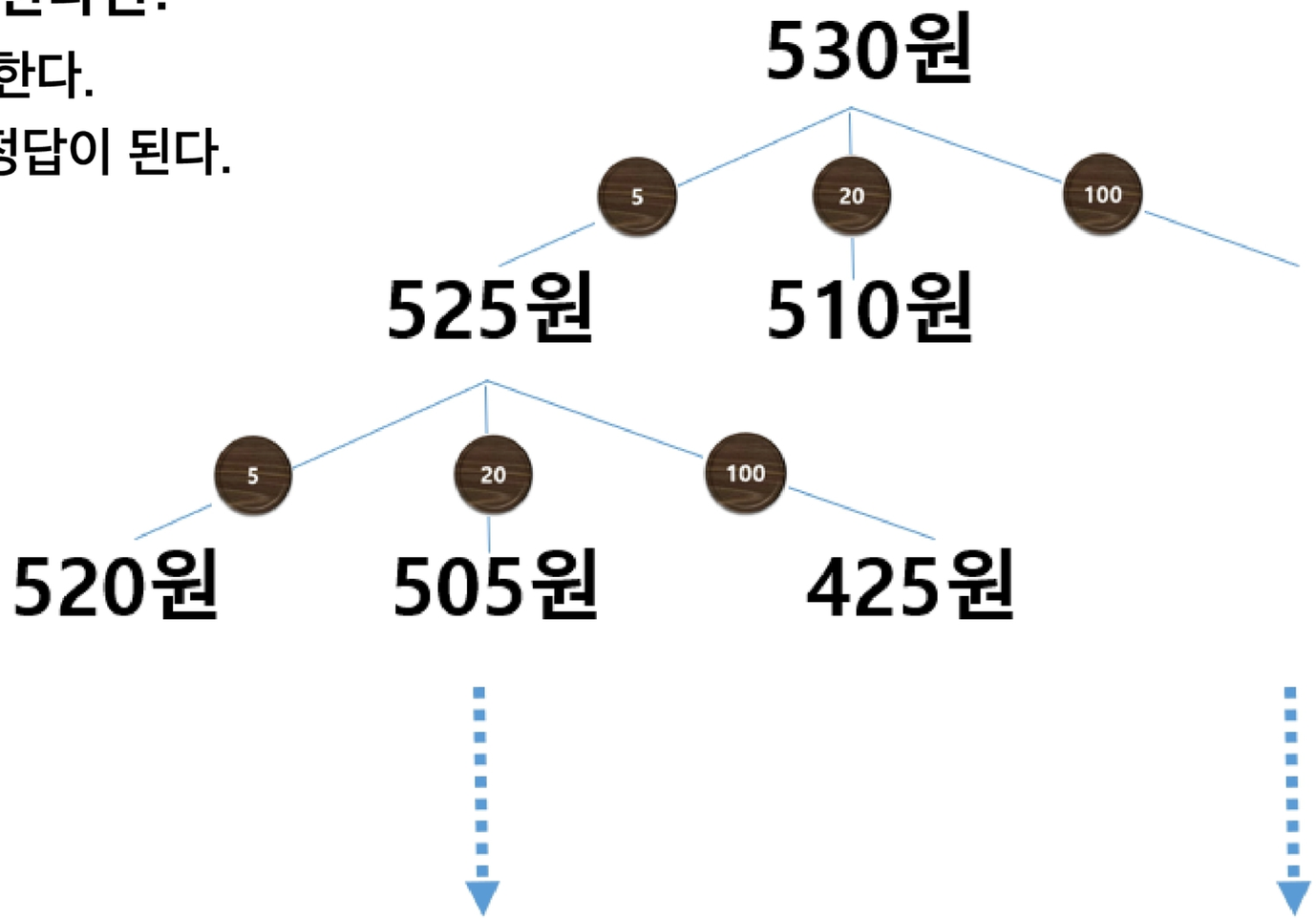
동전이 5, 20, 100 단위로 존재

530원을 거슬러 주기 위해서 사용해야 하는 최소 동전의 수는?

〈그림5_알고리즘 응용_탐욕 알고리즘_동전 교환2〉

동전 교환 문제를 완전탐색으로 해결한다면?

1. 0원이 될 때까지 모든 경우를 다 구한다.
2. 최소 Level이 되는 경우를 찾으면 정답이 된다.



<그림6_알고리즘 응용_탐욕 알고리즘_동전 교환3>

동전 교환 문제를 풀 때, Greedy 알고리즘에서의 예외

만약 100원을 거슬러 주어야 하는 경우, 총 몇 개의 동전이 필요한가?

Greedy로 접근 → 액수가 큰 동전을 먼저 선택하면 4개이다. (70원 하나, 10원 3개)
정답 → 만약 50원 동전만 사용하면 2개

<그림7_알고리즘 응용_탐욕 알고리즘_동전 교환4>

Greedy가 성립하는 경우 vs 성립하지 않는 경우

- 10, 50, 100, 500 처럼 모든 동전이 배수 관계인 경우는 Greedy **적용 가능**
- 10, 50, 70 처럼 모든 동전이 배수 관계가 아닌 경우는 Greedy **적용 불가**

<그림7_알고리즘 응용_탐욕 알고리즘_동전 교환4>

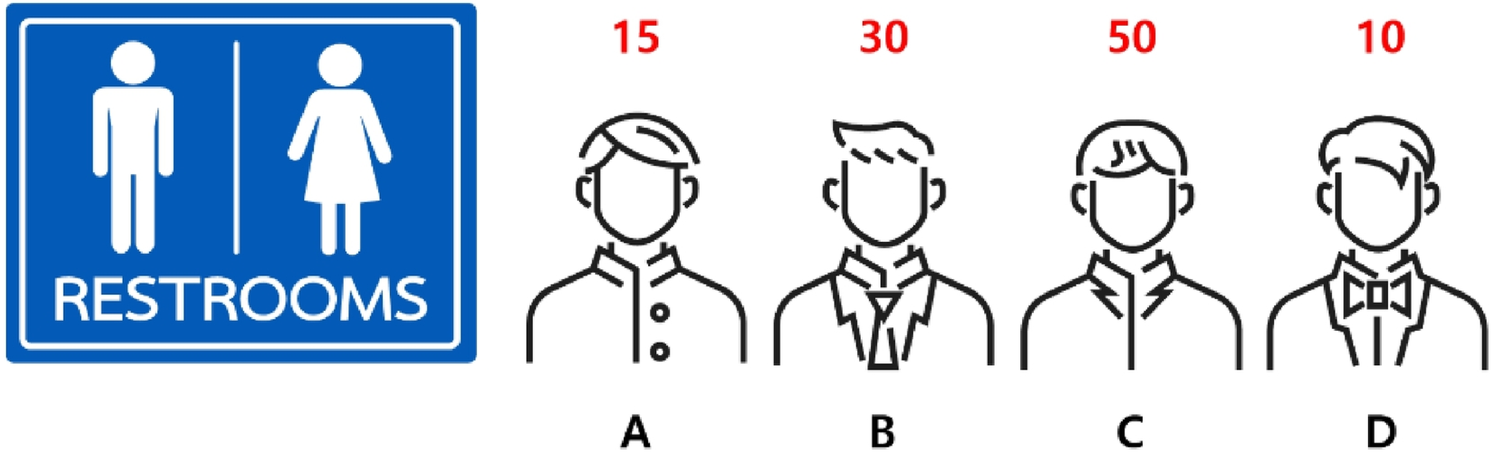
※ 그리디 알고리즘은 쉬워 보이나, 예외 없이 모든 경우가 맞는 규칙인지 아닌지 증명이 어렵습니다.

화장실 문제

기숙사에는 하나의 화장실만 존재한다.

A ~ D 학생은 각자의 평균 화장실 사용 시간이 다음과 같다.

- A : 15분
- B : 30분
- C : 50분
- D : 10분

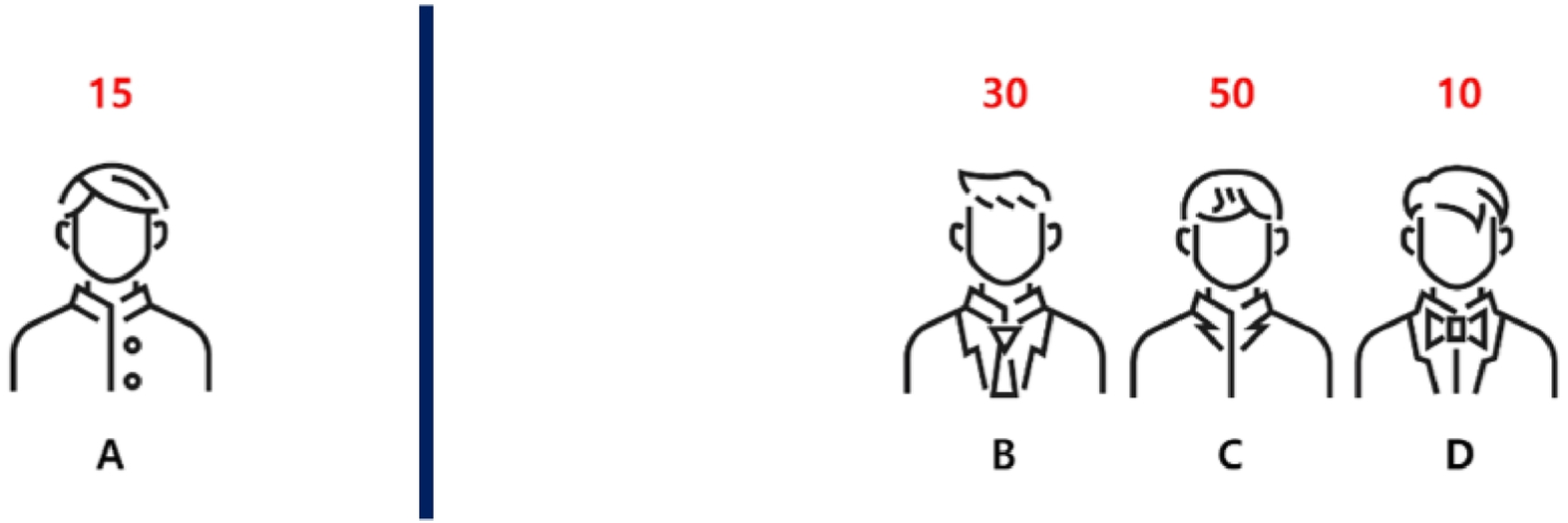


<그림8_알고리즘 응용_탐욕 알고리즘_화장실 문제1>

화장실 문제

만약 A가 먼저 화장실을 이용한다면,
나머지 인원 B, C, D는 대기를 해야 한다.

B, C, D 각자의 대기 시간의 총 합은 15분 * 3 = 45분이다.



<그림9_알고리즘 응용_탐욕 알고리즘_화장실 문제2>

화장실 문제

이후, B가 화장실을 이용하는 경우
나머지 인원 C, D는 대기를 해야 한다.

각자의 대기시간의 누적 합은 45분 + (30분 * 2) = 105분이다.

<그림10_알고리즘 응용_탐욕 알고리즘_화장실 문제3>

화장실 문제

이후, C가 화장실을 이용하는 경우
나머지 D는 대기를 해야 한다.

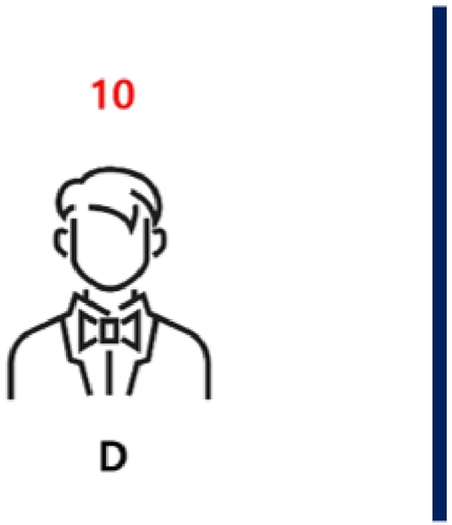
각자의 대기시간의 누적 합은 105분 + (50분 * 1) = 155분이다.

<그림11_알고리즘 응용_탐욕 알고리즘_화장실 문제4>

화장실 문제

마지막으로 D가 화장실을 이용한다.

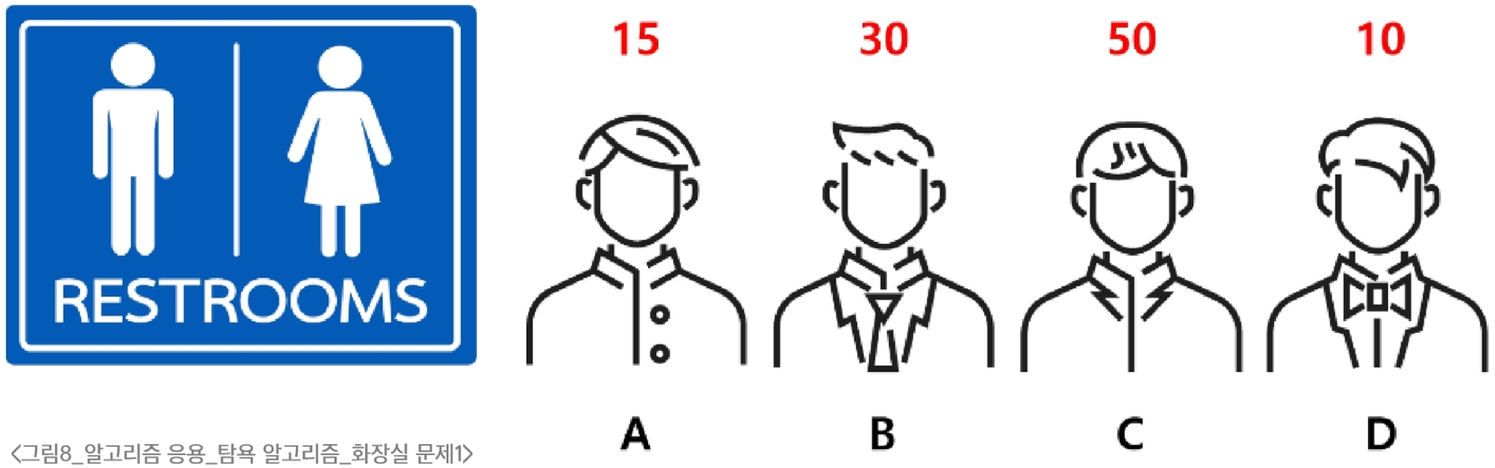
이때는 아무도 기다리지 않기에,
총 대기시간의 누적 합은 155분이다.



<그림12_알고리즘 응용_탐욕 알고리즘_화장실 문제5>

[도전] 화장실 문제를 Greedy로 풀어보자

어떤 기준으로 접근해야 대기시간의 누적 합이 최소가 될지 고민해보고 구현해보자.
(제한시간 5분)
- 코드 : 7.problem_toilet.py



<그림8_알고리즘 응용_탐욕 알고리즘_화장실 문제1>

Knapsack 문제

0-1 Knapsack (물건을 넣거나 뺄 수만 있는 문제)

도둑이 보물들이 있는 창고에 침입하였다.
도둑은 최대 30kg까지 짐을 담아갈 수 있다.

물건의 개수(N) 그리고 물건 별 무게(W)와 가격(P)이 주어질 때,
어떤 물건을 담아야 도둑이 최대 이득을 볼 수 있을지 구하시오.

	무게	값
물건1	5kg	50만원
물건2	10kg	60만원
물건3	20kg	140만원

<표3_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 1>

[도전] 이 문제를 Greedy로 접근해보자. (제한시간 2분)

어떤 기준으로 선택하는 것이 좋을까?

- 물건은 하나씩만 존재한다.
- 최대 30kg까지 짐을 담고자 할 때, 어떤 물건을 담아야 최대로 수익을 낼 수 있는지 찾는 문제

	무게	값
물건1	5kg	50만원
물건2	10kg	60만원
물건3	20kg	140만원

<표3_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 1>

Kg당 가치가 가장 높은 것을 먼저 담으면 안 된다.

0-1 Knapsack을 Greedy로 접근하면 안 되는 예외 케이스가 존재한다.

0-1 Knapsack 문제는 Greedy로 해결할 수 없다.

완전 탐색 혹은 DP로 접근해야 한다.

Kg당 값이 가장 높지만,
물건1을 선택하면 안 된다.

	무게	값	값/kg
물건1	5kg	50만원	10만원/kg
물건2	10kg	60만원	6만원/kg
물건3	20kg	140만원	7만원/kg

<표4_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 2>

※ Kg당 가치가 높은 것을 선택하면 안되는 예시. 정답은 물건2, 3을 선택하는 것이다.

Fractional Knapsack 문제

0-1 Knapsack과 달리, 물건을 원하는 만큼 자를 수 있는 Knapsack 문제.

	무게	값	값/kg
물건1	5kg	50만원	10만원/kg
물건2	10kg	60만원	6만원/kg
물건3	20kg	140만원	7만원/kg

<표4_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 2>

정답은 아니지만, 가방에 담을 수 있는 예시:

물건1에서 5kg + 물건2에서 10kg + 물건3에서 15kg = 50 + 60 +105 = 215만원

Greedy로 접근해도 될지 고민해보자.

Fractional Knapsack 문제 해결 방법

- Greedy가 성립한다.
- kg당 가격이 가장 높은 물건을 최대한 담으면 된다.

- 최대 수익
  - kg당 금액이 가장 높은 물건1, 전체 사용 (50만원)
    + 두 번째로 kg당 금액이 높은 물건3, 전체 사용 (140만원)
    + 세 번째로 kg당 금액이 높은 물건2, 나머지 모두 사용 (5kg만 사용, 30만원)
  = 50 + 140 + 30 = 220만원

	무게	값	값/kg
물건1	5kg	50만원	10만원/kg
물건2	10kg	60만원	6만원/kg
물건3	20kg	140만원	7만원/kg

<표4_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 2>

Fractional Knapsack 소스 코드

```
n = 3
target = 30  # Knapsack KG
things = [(5, 50), (10, 60), (20, 140)]  # (kg, price)

# 정렬 : (price / kg)
total = 0
for kg, price in things:
    per_price = price / kg

    # 만약 가방에 남은 용량이 얼마되지 않는다면,
    # 물건을 잘라 가방에 넣고 끝낸다.
    if target < kg:
        total += target * per_price
        break

    total += price
    target -= kg

print(int(total))
```

	무게	값	값/kg
물건1	5kg	50만원	10만원/kg
물건2	10kg	60만원	6만원/kg
물건3	20kg	140만원	7만원/kg

<표4_알고리즘 응용_탐욕 알고리즘_0-1 Knapsack 2>

- 코드 : 8.problem_fractional_knapsack.py

SAMSUNG SW AI ACADEMY FOR YOUTH

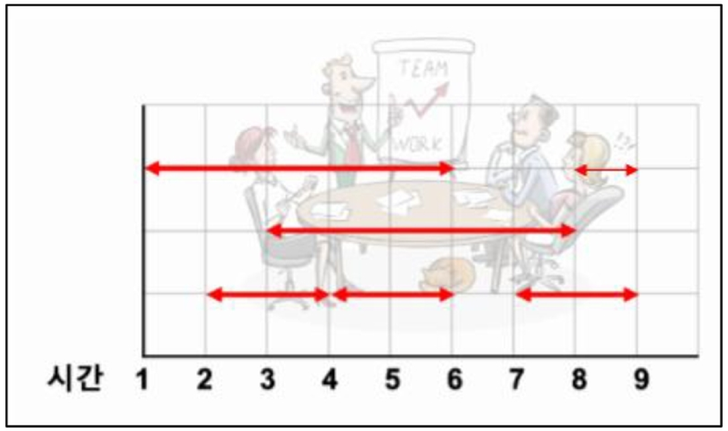
함께가요 미래로!
Enabling People

활동 선택 문제

회의실 배정

회의실이 하나인 회사가 있다.
여러 팀들이 원하는 회의실 예약 시간이 주어질 때,
가능한 많은 회의가 열리기 위해서는 회의들을 어떻게 배정해야 할까?

- 희망 회의 개수, 시작 시간, 종료 시간을 입력 받는다.

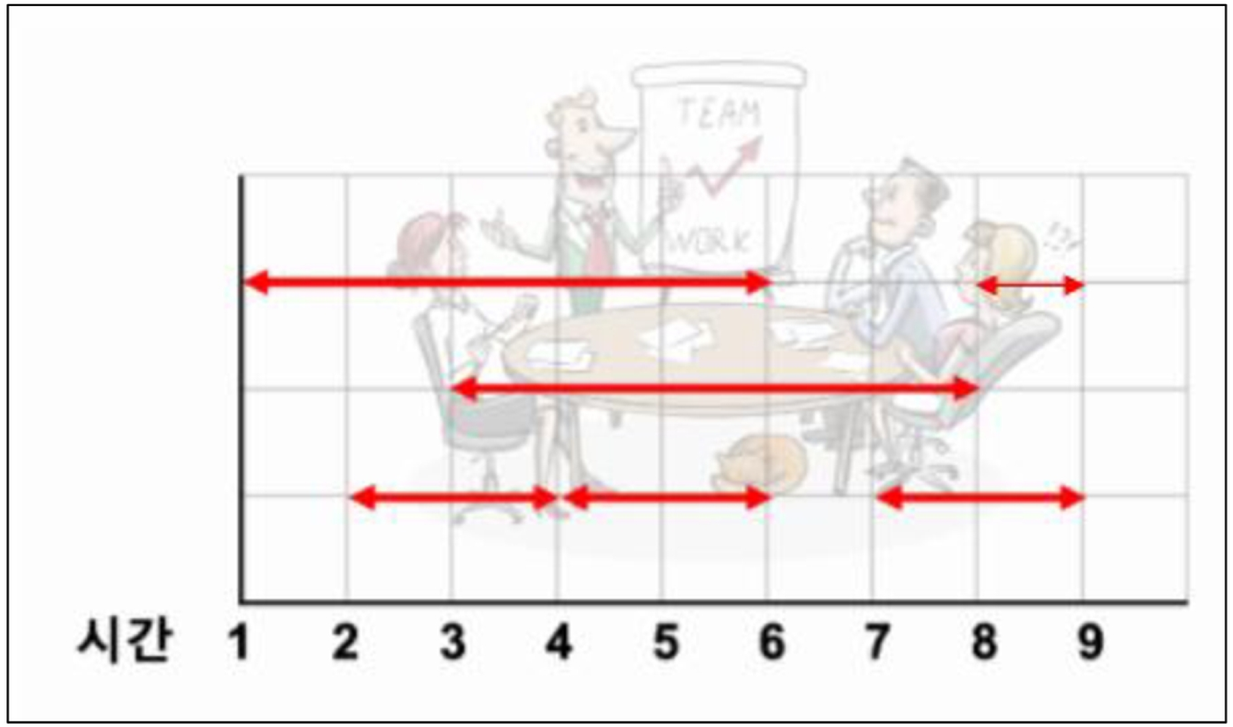


<그림13_알고리즘 응용_탐욕 알고리즘_회의실 배정1>

위와 같이 6개의 희망 회의가 있는 경우,
최대 3번의 회의가 가능하다.

[도전] 무엇을 기준으로 회의를 선택해야 최대한 많은 횟수의 회의가 될지 고민해보자.

이 문제는 Greedy로 가능한 문제이다. (제한시간 2분)



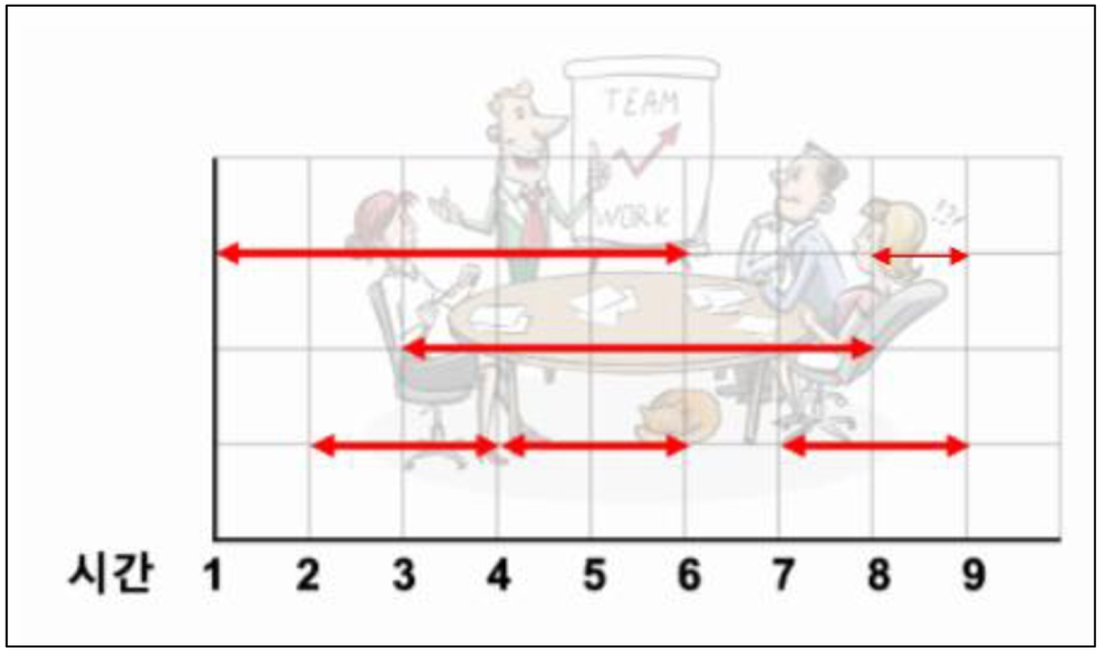
<그림13_알고리즘 응용_탐욕 알고리즘_회의실 배정1>

위와 같이 6개의 희망 회의가 있는 경우,
최대 3번의 회의가 가능하다.

문제 해결 방법

남은 회의 중 종료시간이 가장 빠른 회의를 먼저 선택한다.

- (2, 4), (4,6), (7,9) 또는 (8,9) 회의를 선택하면 3개의 회의를 배정할 수 있습니다.



<그림13_알고리즘 응용_탐욕 알고리즘_회의실 배정1>

다른 예시 - 활동 선택(Activity-selection problem) 문제

총 10개의 회의 요청이 존재한다.

(5,9), (6,10), (8,11), (1,4), (3,5), (1,6), (5,7), (3,8), (2,13), (12,14)

먼저, 종료 시간을 기준으로 10개의 회의들을 오름차순 정렬한다.

(1,4), (3,5), (1,6), (5,7), (3,8), (5,9), (6,10), (8,11), (2,13), (12,14)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a_1															
a_2															
a_3															
a_4															
a_5															
a_6															
a_7															
a_8															
a_9															
a_{10}															

<그림14_알고리즘 응용_탐욕 알고리즘_회의실 배정2>

종료 시간이 가장 빠른 회의를 찾자마자, 확정한다.

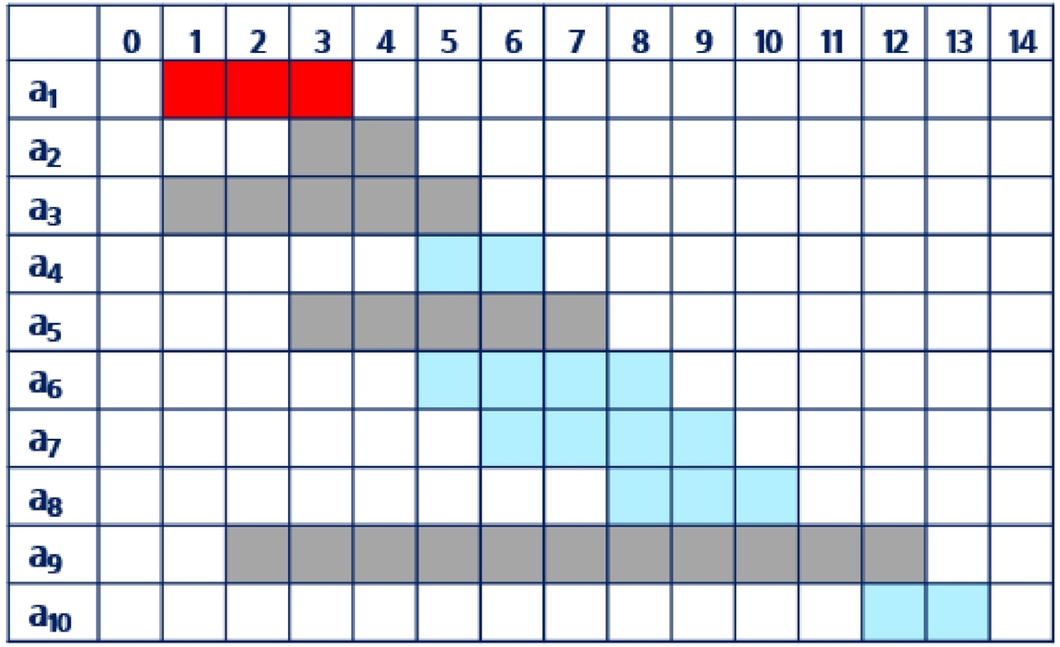
(1,4), (3,5), (1,6), (5,7), (3,8), (5,9), (6,10), (8,11), (2,13), (12,14)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a_1															
a_2															
a_3															
a_4															
a_5															
a_6															
a_7															
a_8															
a_9															
a_{10}															

<그림15_알고리즘 응용_탐욕 알고리즘_회의실 배정3>

이제는 4시 이후부터 회의가 가능하다.

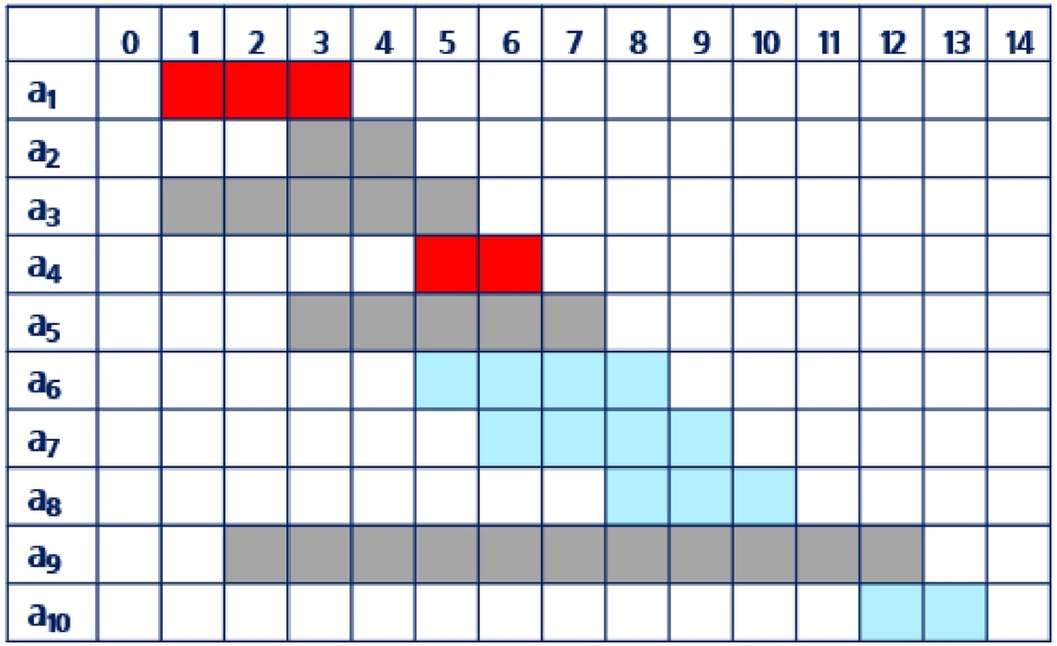
(1,4), ~~(3,5),~~ ~~(1,6),~~ (5,7), ~~(3,8),~~ (5,9), (6,10), (8,11), ~~(2,13),~~ (12,14)



<그림16_알고리즘 응용_탐욕 알고리즘_회의실 배정4>

4시 이후에 가능한 회의 중, 가장 빨리 끝나는 회의를 찾아 확정한다.

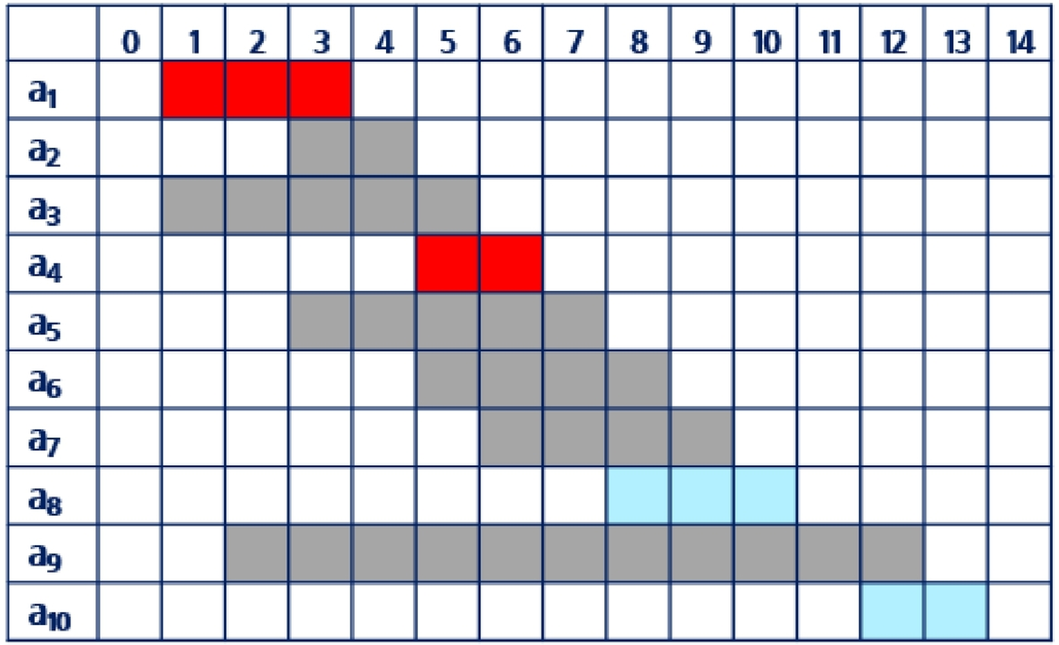
(1,4), ~~(3,5), (1,6),~~ (5,7), ~~(3,8),~~ (5,9), (6,10), (8,11), ~~(2,13),~~ (12,14)



<그림17_알고리즘 응용_탐욕 알고리즘_회의실 배정5>

이제부터 7시 이후로 회의가 가능하다.

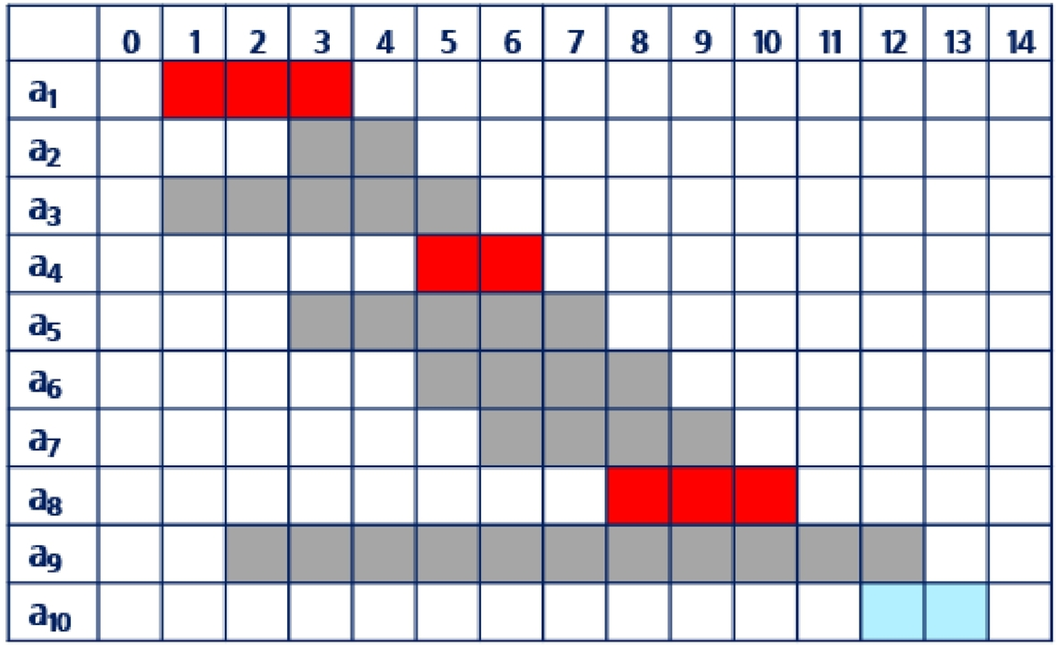
(1,4), ~~(3,5), (1,6),~~ (5,7), ~~(3,8), (5,9), (6,10),~~ (8,11), ~~(2,13),~~ (12,14)



<그림18_알고리즘 응용_탐욕 알고리즘_회의실 배정6>

7시 이후에 가능한 회의 중, 가장 빨리 끝나는 회의를 찾아 확정한다.

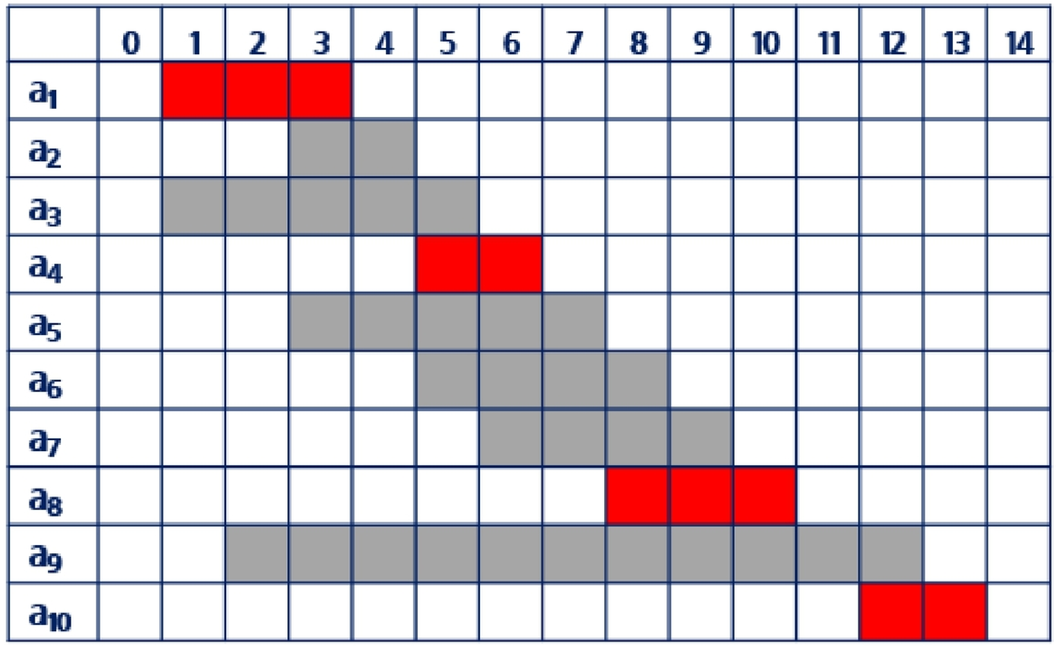
(1,4), ~~(3,5), (1,6),~~ (5,7), ~~(3,8), (5,9), (6,10),~~ (8,11), ~~(2,13),~~ (12,14)



<그림19_알고리즘 응용_탐욕 알고리즘_회의실 배정7>

11시 이후에 가능한 회의 중, 가장 빨리 끝나는 회의를 찾아 확정한다.

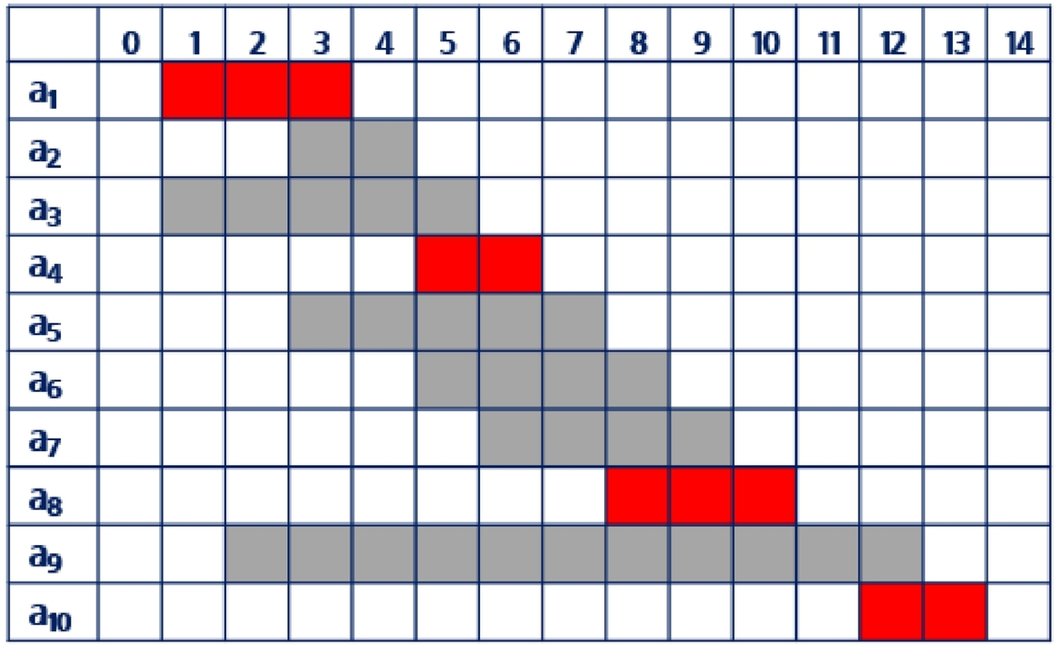
(1,4), ~~(3,5), (1,6),~~ (5,7), ~~(3,8), (5,9), (6,10),~~ (8,11), ~~(2,13),~~ (12,14)



<그림20_알고리즘 응용_탐욕 알고리즘_회의실 배정8>

회의실 문제 전략

1. 끝나는 시간을 기준으로 오름차순 정렬한다.
2. 빠르게 끝나는 회의를 선택하여 확정한다.
3. 이후로 가능한 회의 중, 빠르게 끝나는 회의를 선택하여 확정한다.



<그림20_알고리즘 응용_탐욕 알고리즘_회의실 배정8>

활동 선택 문제(Activity-selection problem)

- 종료 시간이 빠른 순서로 활동들을 정렬한다.
- 첫 번째 활동(A1)을 선택한다.
- 선택한 활동(A1)의 종료시간보다 빠른 시작 시간을 가지는 활동을 모두 제거한다.
- 남은 활동들에 대해 앞의 과정을 반복한다.

```
활동들을 종료 시간 기준으로 오름차순 정렬한다.

선택한 활동 리스트 = 비워둔다
마지막으로 선택한 활동의 종료시간 = 0

for 각 활동 i in 정렬된 활동 리스트:
    if s[i] >= 마지막으로 선택한 활동의 종료시간:
        선택한 활동 리스트에 i 추가
        마지막으로 선택한 활동의 종료시간 = f[i]
```

연습 문제

부분집합의 합 문제 구현하기

- 아래의 10개의 정수 집합에 대한 모든 부분집합 중 원소의 합이 0이 되는 부분집합을 모두 출력하시오.

{-1, 3, -9, 6, 7, -6, 1, 5, 4, -2}

출력 결과

{6, -6}

{-1, 3, -2}

{-9, 7, 4, -2}

…

실습

Greedy

- 5201. 컨테이너 운반
- 5202. 화물도크
- 5203. 베이비진 게임

부분집합

- 4837. 부분집합의 합

추가 연습

- 4012. [모의 SW 역량테스트] 요리사

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 Greedy 알고리즘이 유효하기 위한 조건은?

a) 항상 최악의 경우를 고려해야 한다.
b) 최적 부분 구조와 탐욕 선택 속성이 있어야 한다.
c) 반드시 완전탐색을 포함해야 한다.
d) 분할 정복과 함께 사용해야 한다.

2 다음 중 모든 부분집합을 구하는 일반적인 방법은?

a) 재귀 호출을 통한 분할 정복
b) Binary Counting
c) BFS 탐색
d) 연결 리스트 활용

3 다음 중 Greedy 알고리즘으로 풀 수 없는 문제는?

a) 회의실 배정 문제
b) Fractional Knapsack 문제
c) 0-1 Knapsack 문제
d) 동전 거스름돈 문제 (단위: 500, 100, 50, 10)

4 아래 코드에서 출력되는 결과는?

```
def is_valid(start, end, current_end):
    return start >= current_end

print(is_valid(5, 6, 4))
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 아래 코드에서 출력되는 결과는?

```
items = [(10, 60), (20, 100), (30, 120)]
items.sort(key=lambda x: x[1]/x[0], reverse=True)
print(items[0])
```

6 Fractional Knapsack 문제에서 사용하는 전략은?

a) 무게 순 정렬
b) 가치 순 정렬
c) 단위 무게당 가치순 정렬
d) ID 순 정렬

7 회의실 배정 문제에서 가장 중요한 정렬 기준은?

a) 회의 시작 시간 오름차순
b) 회의 시간 길이 내림차순
c) 회의 종료 시간 오름차순
d) 회의 시간 길이 오름차순

8 아래 코드에서 val에 저장된 값은?(실수로 표현)

```
weight = 15
items = [(10, 60), (20, 100), (30, 120)]
items.sort(key=lambda x: x[1]/x[0], reverse=True)
val = 0
val += items[0][1] * (weight / items[0][0])
```

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 Greedy 알고리즘으로 풀 수 없는 예는?

a) 10, 50, 100 단위 동전으로 1730원 거슬러주기
b) 5, 50, 70 단위 동전으로 100원 거슬러주기
c) 종료시간 기준으로 회의 선택하기
d) Fractional Knapsack에서 단위가치 순으로 물건 선택하기

10 아래 코드에서 sum의 값은?

```
arr = [7, 3, 2, 6]
arr.sort(reverse=True)
sum = 0
for i in range(2):
    sum += arr[i]
```

11 coins의 동전으로 6원을 거슬러 주는 코드에서 cnt의 값은? 이는 최적해인가?

```
coins = [1, 3, 4]
amount = 6
cnt = 0
for coin in reversed(coins):
    cnt += amount // coin
    amount %= coin
```

12 Binary Counting 을 통해 집합 {A, B, C}의 부분집합 101에 해당하는 것은?

a) {A, B}
b) {A, C}
c) {B, C}
d) {C}

확인문제

정답 및 해설

1. b) 최적 부분 구조와 탐욕 선택 속성이 있어야 한다. 2. b) Binary Counting 3. c) 0-1 Knapsack 문제 4. True 5. (10, 60) 6. c) 단위 무게당 가치순 정렬 7. c) 회의 종료 시간 오름차순 8. 90.0 9. b) 5, 50, 70 단위 동전으로 100원 거슬러주기 10. 13 11. 3, 아님 12. b) {A, C}

1. Greedy 알고리즘이 유효하기 위해서는 최적 부분 구조와 탐욕 선택 속성이 있어야 합니다.

2. 부분집합을 구하기 위해서 Binary Counting을 사용하는 것이 가장 일반적입니다. 학습용으로 완전 탐색을 선택할 수도 있습니다.

3. 0-1 Knapsack 문제는 Greedy 알고리즘으로 접근하면 안되는 예외 케이스가 존재하기 때문에 풀 수 없습니다. 동전 거스름돈 문제는 모든 동전이 배수 관계가 성립하면 Greedy 알고리즘으로 접근 가능합니다.

4. 회의실 배정 문제에서 다음 회의를 선택할 수 있는지 판단하는 함수입니다. Start(5)가 현재 회의의 종료시간인 current_end(4)보다 크거나 같으므로 회의를 배정할 수 있습니다. 즉, True가 출력됩니다.

5. 각 아이템의 단위 무게당 가치는 [6.0, 5.0, 4.0]이고, 가장 높은 6.0을 가진 (10, 60)이 정렬 후 첫 번째가 됩니다.

6. Fractional Knapsack 문제에서는 단위 무게당 가치순 정렬을 사용합니다.

7. 회의실 배정 문제에서는 회의 종료 시간 오름차순을 기준으로 정렬합니다.

8. 가장 높은 단위가치: (10, 60) -> 6.0
15kg 중 10kg만 채우면 되므로 60 * (15 / 10) = 90.0

9. 5, 50, 70 단위 동전으로 100원 거슬러주기는 모든 동전이 배수 관계가 성립하지 않기 때문에 Greedy 알고리즘으로 풀 수 없습니다. 동전의 수가 최소가 되기 위한 답은 50원 동전 2개이지만, Greedy를 활용하면 70원 1개와 5원 6개인 총 7개의 동전을 사용하게 됩니다.

10. 탐욕적으로 가장 큰 값 2개 선택(7, 6)하여 합한 값은 7 + 6 = 13입니다.

11. 동전 [4, 3, 1]에서 6 // 4 = 1, 남은 2 // 1 = 2로 3이 답입니다. 그러나 최적의 해는 3 + 3 = 6으로 2개를 사용하는 것입니다. 따라서 해당 코드는 항상 그리디가 정답이 아님을 보여주는 예시입니다.

12. 집합 {A, B, C}에서 바이너리 카운팅을 활용해 구한 101의 부분집합은 {A, C}입니다.

활동정리

핵심키워드

개념	설명	예시
탐욕 알고리즘 (Greedy)	현재 상황에서 가장 좋아 보이는 해를 선택하는 알고리즘 기법	동전 교환, 회의실 배정 문제 등
최적 부분 구조	문제의 최적해가 부분 문제의 최적해로 구성되는 성질	부분 최적들이 전체 최적으로 연결될 수 있음
탐욕 선택 속성	매 단계에서의 국지적 최적 선택이 전체 최적해를 이끄는 성질	가장 빨리 끝나는 회의를 선택하면 전체 최적 회의 배정 가능
단위 무게 당 가치	가치 ÷ 무게로 계산되며, 일부 문제에서 선택 기준으로 활용	(value=60, weight=10) → 6.0
부분집합 생성 방법	집합의 모든 부분집합을 나열하는 방법	Binary Counting, 재귀 백트래킹 등
조합(Combination)	서로 다른 n개 중 r개를 순서 없이 고르는 방법	{A, B, C} 중 2개 선택: AB, AC, BC
Binary Counting	이진수의 0/1을 이용해 원소의 선택/비선택을 나타내는 부분집합 생성 기법	0b101 → 첫 번째와 세 번째 원소 선택

〈표5_알고리즘 응용_탐욕 알고리즘_핵심키워드〉

활동정리

오늘 공부한 내용 요약 및 정리

- **회의실 배정 전략**

  - 회의를 종료 시간이 빠른 순으로 정렬하고, 앞선 회의와 겹치지 않는 회의만 선택하면 최대 회의 수를 배정할 수 있다.

- **Fractional Knapsack의 핵심 전략**

  - 단위 무게당 가치가 가장 높은 물건부터 담는 것이 항상 최적의 해를 보장하며, 물건을 쪼갤 수 있다는 점이 핵심이다.

- **0-1 Knapsack에서 탐욕의 한계**

  - 탐욕적으로 단위가치가 높은 것부터 선택하면 오답이 나올 수 있으며, 이 문제는 백트래킹이나 동적 계획법이 필요하다.

활동정리

오늘 공부한 내용 요약 및 정리

- **탐욕 알고리즘의 실패 사례**
  - 동전 단위가 일정하지 않을 때, 큰 단위부터 선택하면 최소 개수가 되지 않는 경우가 발생한다.

- **탐욕 알고리즘의 특징**
  - 탐욕 알고리즘의 핵심은 '어떤 기준으로 선택할 것인가'에 달려 있으며, 문제마다 적절한 기준을 설정하는 것이 중요하다.

- **탐욕과 완전탐색의 비교**
  - 완전탐색은 항상 정답을 보장하지만 비효율적이고, 탐욕은 빠르지만 조건이 맞아야 정답을 보장한다.

활동정리

'컴퓨터가 말하는 탐욕 알고리즘을 이해해 봅시다!'

1. 문제를 풀 때마다 [ ] 선택을 합니다. 왜냐하면 매 순간의 선택이 결국 전체 문제의 [ ]라고 믿기 때문입니다. 이런 방식이 잘 통하려면 탐[ ] 속성이 있어야 합니다.
2. 항상 [ ] 기준을 대상으로 정렬하고, 그중에서 [ ]만을 반복합니다. 하지만 매번 성공하는 것은 아닙니다. 어떤 문제는 [ ]를 만족하지 않으면 틀릴 수 있습니다.

활동정리

'컴퓨터가 말하는 탐욕 알고리즘을 이해해 봅시다!'

1. 문제를 풀 때마다 가장 좋아 보이는 선택을 합니다. 왜냐하면 매 순간의 선택이 결국 전체 문제의 최적 해 라고 믿기 때문입니다. 이런 방식이 잘 통하려면 탐욕 선택 속성이 있어야 합니다.
2. 항상 명확한 기준을 대상으로 정렬하고, 그 중에서 즉각적인 선택만을 반복합니다. 하지만 매번 성공하는 것은 아닙니다. 어떤 문제는 최적 부분 구조를 만족하지 않으면 틀릴 수 있습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다